

Enterprise Tiny Agent Framework

Supplementary Engineering Guide

Version: 1.0

Purpose

This document expands the core Agent Framework Design Guide and provides implementation guidance, engineering standards, design rationale, and operational best practices.

This document focuses on:

- How to build agents
- How to build tools
- How to select models
- How to manage context
- How to avoid common agent failures
- How to scale the platform

The target environment is:

```
Java 21+  
Spring Boot  
DuckDB  
Apache Calcite  
PostgreSQL  
Apache Airflow  
Local LLMs  
Hosted LLMs
```

Chapter 1

Capability Driven Agent Design

Why Capability Matters

The biggest failure in AI projects is designing agents around technologies.

Example:

DuckDB Agent
Filesystem Agent
PostgreSQL Agent

These are implementation details.

Users never ask:

Ask the DuckDB Agent

Users ask:

Find the subscriber usage dataset.

Explain why my job failed.

Generate a reconciliation query.

Capabilities remain stable.

Technologies change.

Capability Discovery Process

Before building an agent answer:

Question 1

What business problem is being solved?

Example:

Find datasets.

Question 2

How will success be measured?

Example:

Correct dataset found.

Question 3

Which tools are required?

Example:

Metadata Search Tool
Lineage Tool

Question 4

Can another existing agent already solve it?

If yes:

Do not create another agent.

Chapter 2

Agent Lifecycle Deep Dive

Initialize Phase

Purpose:

Create execution context.

Activities:

Generate Request ID
Load Session

```
Load Agent Definition
Load Configuration
```

Output:

```
AgentExecutionContext
```

Validate Phase

Purpose:

Reject invalid requests early.

Examples:

```
Missing dataset name
Missing query
Unauthorized action
```

Never allow tool execution before validation.

Context Building Phase

Purpose:

Gather relevant information.

Examples:

```
Metadata lookup
Previous session context
User preferences
Recent tool results
```

Do not retrieve everything.

Retrieve only what is required.

Planning Phase

Purpose:

Determine actions.

Example:

```
User :
Generate SQL for subscriber usage

Plan:

1 Search Metadata
2 Validate Columns
3 Generate SQL
```

A plan should be explainable.

If you cannot explain the plan, it is too complex.

Execution Phase

Purpose:

Perform work.

Execution should be:

```
Deterministic
Observable
Recoverable
```

Every tool invocation should be logged.

Response Generation Phase

Purpose:

Convert execution results into user output.

Never allow response generation to re-execute tools.

Response generation should only transform existing results.

Chapter 3

Tool Engineering Standards

Tool Design Rules

Each tool should perform one operation.

Bad:

```
MetadataTool
```

Good:

```
SearchDatasetTool
```

```
GetDatasetTool
```

```
GetLineageTool
```

```
GetColumnTool
```

Tool Input Contract

Every tool should define:

```
ToolRequest
```

Example:

```
public record SearchDatasetRequest(  
    String searchText,
```

```
Integer limit
) {}
```

Tool Output Contract

Every tool should define:

```
ToolResult
```

Example:

```
public record DatasetSearchResult(  
    List<Dataset> datasets  
) {}
```

Never return free text.

Tool Versioning

Version tools independently.

Example:

```
search-dataset-v1  
  
search-dataset-v2
```

Agents should not break when tools evolve.

Chapter 4

Model Selection Strategy

Principle

Choose the smallest model capable of performing the task.

Classification

Typical workloads:

Intent Classification

Recommended:

Tiny Local Model

Metadata Search

Recommended:

2B - 4B

SQL Generation

Recommended:

4B - 8B

Research

Recommended:

8B - 14B

Planning

Recommended:

14B+

Escalation Strategy

Example:

```
Try Local 4B  
  
Confidence < 0.80  
  
Escalate to 14B  
  
Confidence < 0.80  
  
Escalate to Hosted Model
```

Chapter 5

Context Engineering

Most Agent Failures Are Context Failures

Not model failures.

Example:

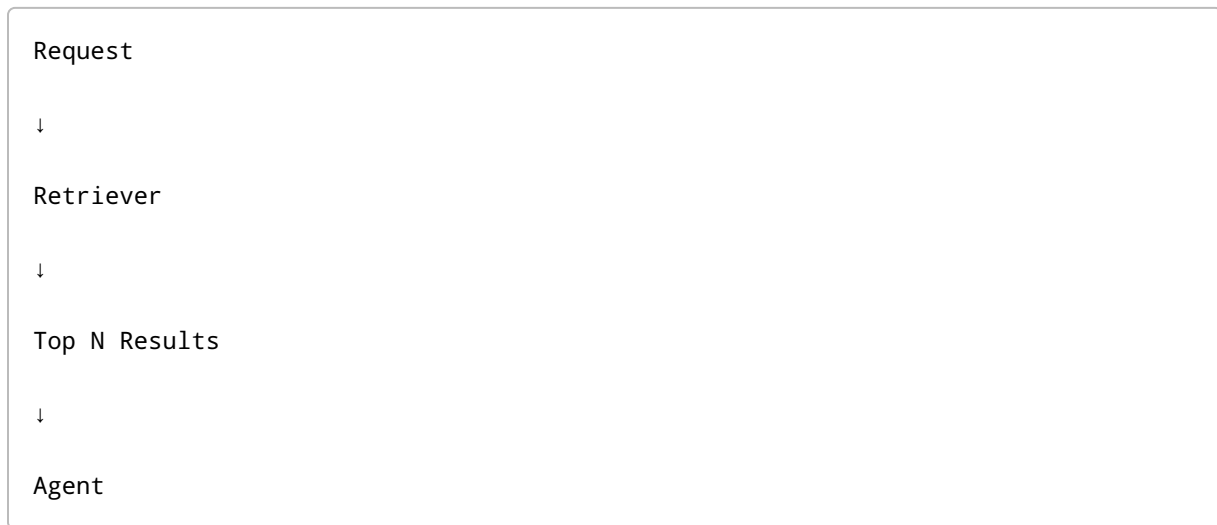
Bad:

```
Entire catalog  
Entire schema repository  
Entire documentation
```

Good:

```
Relevant schema  
  
Relevant datasets  
  
Relevant documents
```

Retrieval Pattern



This should be the default pattern.

Context Budget

Recommended:

Request	10%
Retrieved Context	70%
Instructions	20%

Never invert this ratio.

Chapter 6

Agent Memory

Memory Types

Request Memory

Exists for one request.

Session Memory

Exists for one conversation.

Persistent Memory

Exists across sessions.

Use sparingly.

What Not To Store

Avoid:

Entire conversations

Tool outputs

Large documents

Store summaries.

Chapter 7

Observability

Every Agent Must Be Observable

Track:

Request ID

Agent

Model

Execution Time

Tool Calls

Failures

Tokens

Cost

Audit Event Schema

AgentEvent

Fields:

eventId

requestId

agentId

timestamp

eventType

payload

Chapter 8

Agent Testing

Unit Testing

Test:

Validation

Planning

Tool Selection

Response Generation

Mock tools.

Integration Testing

Test:

Agent + Tools

Evaluation Testing

Maintain datasets:

Input

Expected Output

Expected Tool Usage

Run continuously.

Chapter 9

Security

Principle Of Least Privilege

Each agent gets access only to required tools.

Example:

Metadata Agent

Allowed:

Metadata Tools

Forbidden:

Restart Job Tool

Dangerous Operations

Require approval workflow.

Examples:

Delete Dataset

Restart Production Job

Modify Rules

Chapter 10

Recommended Initial Agents

Metadata Agent

Capability:

Dataset Discovery

Tools:

SearchDatasetTool

GetDatasetTool

GetLineageTool

SQL Agent

Capability:

Generate SQL

Tools:

Metadata Tool

Calcite Validator

DuckDB Explain

Operations Agent

Capability:

Manage Jobs

Tools:

Get Job Status

Get Job Logs

Restart Job

Documentation Agent

Capability:

Explain Platform

Tools:

Document Search

Metadata Search

Research Agent

Capability:

Collect Information

Tools:

Web Search

Document Reader

Summarizer

Chapter 11

Anti-Patterns

Avoid:

One agent doing everything

Avoid:

Agents directly calling agents

Avoid:

Massive prompts

Avoid:

100-tool agents

Avoid:

Using large models for every request

Avoid:

Storing everything as memory

Final Recommendation

Start with:

Metadata Agent

SQL Agent

Operations Agent

Build:

Tool Registry

Model Router

Agent Runtime

Audit Framework

before introducing:

MCP

Multi-Agent Systems

Autonomous Planning

Complex Memory

A stable foundation will outperform a sophisticated architecture built too early.